HANDS-ON TECHNICAL EVALUATION · PRACTICAL ASSIGNMENT

INSTRUCTOR
Lipton Ahammed

System & Application Observability

Build and demonstrate a working observability platform for real Ubuntu host metrics, HTTP application metrics, Kubernetes workload logs, Grafana visualizations, and email-based alerting.

Ubuntu 26.04	MicroK8s	12 Nov 2026	ZIP or GitHub
TARGET SYSTEM	LOCAL KUBERNETES	SUBMISSION DATE	REQUIRED DELIVERY

Offline-friendly assessment brief · Print directly or save as PDF

Print / Save as PDF

Candidate information

Complete these fields before submitting your assessment.

CANDIDATE NAME
Full name

CANDIDATE ID
Candidate or employee ID

ASSESSMENT DATE
mm/dd/yyyy

GITHUB URL (IF APPLICABLE)
https://github.com/...

Assessment scenario. You are responsible for monitoring a small application environment on your Ubuntu laptop. The solution must demonstrate **metrics from the actual Ubuntu host and a real HTTP application**, **searchable logs from a running Kubernetes cluster**, and **an alert email received after a controlled test**. Use real, changing data. Screenshots alone are not a substitute for a reproducible implementation.

What you must deliver

The assessment is considered technically complete only when all three outcomes can be demonstrated.

01
Live metrics
Prometheus scrapes Ubuntu host and application metrics; Grafana displays CPU, RAM, filesystem, network, traffic, errors, and latency.

02
Kubernetes logs
A real MicroK8s cluster runs a workload. Grafana Alloy sends its pod logs to Loki, where they can be filtered in Grafana.

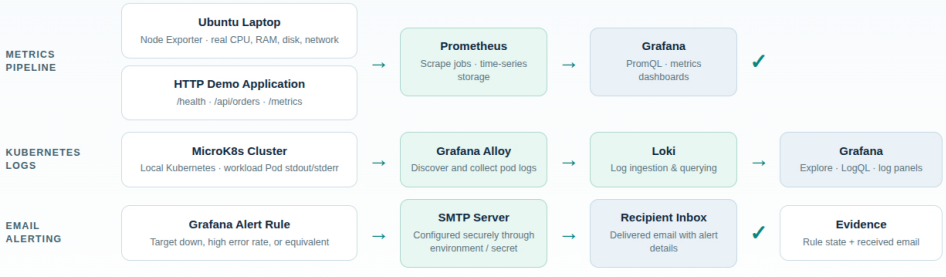
03
Email alerting
A meaningful alert is triggered, Grafana sends a notification through configured SMTP, and a message is received in a test inbox.

Expected architecture

Reference architecture for the assessment. Equivalent working arrangements are acceptable if the same end-to-end flows are shown.

THREE CONNECTED OPERATIONAL FLOWS

Metrics · Logs · Notifications



Deployment boundary: Run Prometheus, Grafana, Loki, Node Exporter, and the instrumented HTTP app with Docker Compose on Ubuntu. Run a separate local MicroK8s cluster for the log-producing workload and deploy Alloy inside the cluster. Ensure Alloy can reach the Loki ingestion endpoint across these environments; document your networking method. Other equivalent layouts are permitted if the required signals and evidence are preserved.

Practical tasks & acceptance requirements

Perform the following tasks on your own machine. For each task, show a working result to the evaluator and include evidence in your submission.

01 ENVIRONMENT
Prepare Ubuntu, Docker and Kubernetes tooling

- Use Ubuntu 26.04 with native Docker Engine and Docker Compose; install **MicroK8s** and configure `kubectl` and Helm (or use their MicroK8s-provided commands).
- Create a logical repository structure for application code, Compose, Prometheus, Loki, Kubernetes manifests, Alloy, Grafana and evidence.
- Verify the toolchain versions and record the setup commands needed to reproduce the environment.

Show: Ubuntu version, `docker compose version`, `microk8s status --wait-ready`, `microk8s kubectl get nodes` and directory structure.

02 DEPLOYMENT
Run the metrics and logging backend

- Create a `compose.yaml` stack containing Prometheus, Grafana, Node Exporter, Loki and your HTTP application.
- Configure named volumes for durable data, appropriate restart behavior, service discovery, and configuration through files or environment variables.
- Make Node Exporter read the actual Ubuntu host metrics using suitable read-only mounts and host-aware options.
- Keep observability endpoints private to the laptop. Do not publish Grafana, Loki, Prometheus or exporters on a public interface.

Show: `docker compose config`, `docker compose ps`, running services, and persistence after a restart.

03 APP INSTRUMENTATION

Create a measurable HTTP application

- Build a working Order API (Flask, FastAPI, Node.js, etc.) or equivalent using `GET /health`, `POST /api/orders` and `GET /metrics`.
- Export a request counter, latency histogram, failure/success information, and at least one business-event counter using a supported Prometheus client library.
- Handle normal and deliberately failed requests safely. Use bounded metric labels and clear metric names and units.
- Generate traffic at different rates and verify that counters, histogram buckets and errors change.

Show: Successful and failed HTTP responses, `/metrics` output with custom metrics, and before/after metric values.

04 PROMETHEUS METRICS

Collect and analyze Ubuntu and application telemetry

- Create separate Prometheus scrape jobs for Node Exporter and the application, using a suitable interval (for example, 15 seconds).
- Confirm both targets report **UP** and measure actual CPU, RAM, disk, network, uptime, request count, failure rate and latency.
- Write and explain PromQL expressions for CPU %, RAM %, application requests/second, errors/failure rate and p95 request latency.
- Compare at least two host readings against Linux commands such as `free`, `df` and `uptime`.

Show: Prometheus Targets with both jobs UP; five working PromQL expressions; and two Ubuntu cross-checks.

05 KUBERNETES & LOKI

Create a cluster and visualize real MicroK8s pod logs

- Start and verify a local **MicroK8s** cluster. Deploy at least one application or log-generating workload using a Kubernetes Deployment and a dedicated namespace.
- Ensure the workload writes informative entries to stdout and stderr, including normal events and an intentional error scenario.
- Install and configure **Grafana Alloy** in the cluster (Helm chart or Kubernetes manifests) to discover MicroK8s pod logs and forward them to **Loki**.
- Connect Loki as a Grafana data source. Use Grafana Explore and LogQL to filter logs by namespace, pod or container; show recent errors and timestamps.
- Document how traffic reaches Loki from the cluster and ensure the displayed logs originate from the live Kubernetes workload, not static sample files.

Show: `microk8s kubectl get nodes`, `microk8s kubectl get pods -A`, `microk8s kubectl logs` from the workload, healthy Alloy, Loki ingestion and a matching Grafana log query.

06 VISUALIZATION

Build an operational Grafana dashboard

- Configure both Prometheus and Loki data sources in Grafana.
- Build at least eight useful metrics panels: CPU %, RAM %, disk utilization, network throughput, requests/second, error rate, p95 latency and business-event count.
- Demonstrate Kubernetes logs in Grafana Explore; optionally add a dedicated log panel or a second Kubernetes logs dashboard.
- Use meaningful chart titles, query legends, appropriate units and time ranges. Save and export all dashboard JSON files.

Show: At least eight live metrics panels, searchable MicroK8s pod logs, and exported dashboard JSON.

07 NOTIFICATION DELIVERY

Configure and test email alerting

- Configure outbound SMTP for Grafana using environment variables or a secret mechanism. Do not hardcode SMTP credentials in source control.
- Create a Grafana email contact point and notification policy. Configure one actionable alert rule, such as application scrape target down or sustained high error rate.
- Trigger the alert safely, allow its evaluation and notification to run, and verify that an email actually arrives at the designated test inbox.
- Explain the alert threshold, evaluation duration, notification recipient and recommended operator action.

Show: Alert rule/configuration, firing state or alert history, contact-point configuration (credentials hidden), and a received test email with relevant timestamp and subject.

08 DEMONSTRATION & HANDOVER

Demonstrate the full pipeline and submit the project

- Show a normal request, a higher traffic rate, and a controlled failure. Explain the corresponding change in application metrics and relevant Kubernetes logs.
- Explain the architecture: metrics ingestion, Kubernetes log collection, dashboard queries and SMTP alert delivery. Mention one limitation or improvement.
- Include reproducible setup, verification, cleanup steps and all required configuration files in a README.
- **Submit your complete project as a .zip archive OR provide an accessible GitHub repository link.** A screenshot-only or document-only submission is not acceptable.

Show: End-to-end live demonstration, the project archive or repository URL, all required evidence and a clear README.

Assessment rules & boundaries

- Use actual host and application metrics, and actual container logs from the Kubernetes cluster. Avoid invented, static or mocked observability outputs.
- Keep the infrastructure local unless the evaluator expressly approves remote access. Expose ports only as required for local testing.
- Do not put API tokens, passwords, personal information, SMTP credentials or recipient secrets in GitHub, the ZIP or screenshots. Use an `.env.example` and a `.gitignore`.
- Equivalent implementation methods are allowed when they reproduce all required behavior. Document any deviations from the reference architecture.
- The project must be the candidate's own implementation. Official documentation and reference examples may be consulted.

Project submission — ZIP or GitHub

Project submission — ZIP or GitHub

Submit by **12 Nov 2026**. Deliver one complete, reproducible project. Choose either a ZIP file or an accessible GitHub repository (public or shared with the evaluator).

Mandatory delivery: `candidate-ID_observability.zip` **OR** a GitHub repository URL. Include source code, infrastructure configuration, exported dashboards, tests, README and evidence. Do not submit only screenshots or a PDF report.

Evidence checklist

1. Tool versions; `docker compose ps`, `microk8s status` and `microk8s kubectl get pods -A`.
2. Prometheus Targets: Ubuntu and application jobs UP.
3. Application `/metrics` plus traffic and failure test.
4. Five PromQL results and Ubuntu system cross-checks.
5. Grafana metrics dashboard (at least eight panels).
6. MicroK8s workload logs in both `microk8s kubectl logs` and Grafana Loki Explore.
7. Grafana alert state and the received email alert (credentials and sensitive details hidden).
8. README with setup, configuration, access, test and cleanup instructions.

Suggested project structure

```
candidate-ID_observability/
├── README.md
├── compose.yaml
├── .env.example
├── .gitignore
├── app/
│   ├── Dockerfile
│   └── ... source files
├── prometheus/
│   └── prometheus.yml
├── loki/
│   └── config.yml
├── kubernetes/
│   ├── namespace.yaml
│   ├── workload.yaml
│   └── alloy-values.yaml
├── grafana/
│   ├── dashboards/
│   └── provisioning/
├── alerting/
│   └── configuration-notes.md
├── scripts/
│   └── traffic-test.sh
└── evidence/
    ├── compose-and-cluster.png
    ├── prometheus-targets.png
    ├── grafana-metrics.png
    ├── kubernetes-logs.png
    └── email-alert.png
```

Final review checklist

The evaluator should verify the following observable outcomes during the live demonstration.

AREA	EXPECTED WORKING RESULT	PROOF
Host & app metrics	Both Prometheus targets UP; live measurements respond to traffic and host activity.	Prometheus + terminal
Kubernetes logging	Cluster workload logs appear in Loki and can be searched in Grafana with Kubernetes labels.	`microk8s kubectl logs` + Grafana Explore
Visualizations	At least eight accurate metrics panels; logs accessible via Grafana.	Grafana dashboard + LogQL
Email alert	Configured rule fires under test and delivers email to recipient inbox.	Rule history + received email
Reproducibility	Complete source and configs provided through ZIP or GitHub with safe setup instructions.	Project archive/repository + README

Official documentation

Docker Compose · MicroK8s documentation · Kubernetes · Prometheus · Node Exporter · Grafana Alloy: Kubernetes logs · Grafana Loki · Grafana email contact points · Grafana SMTP configuration